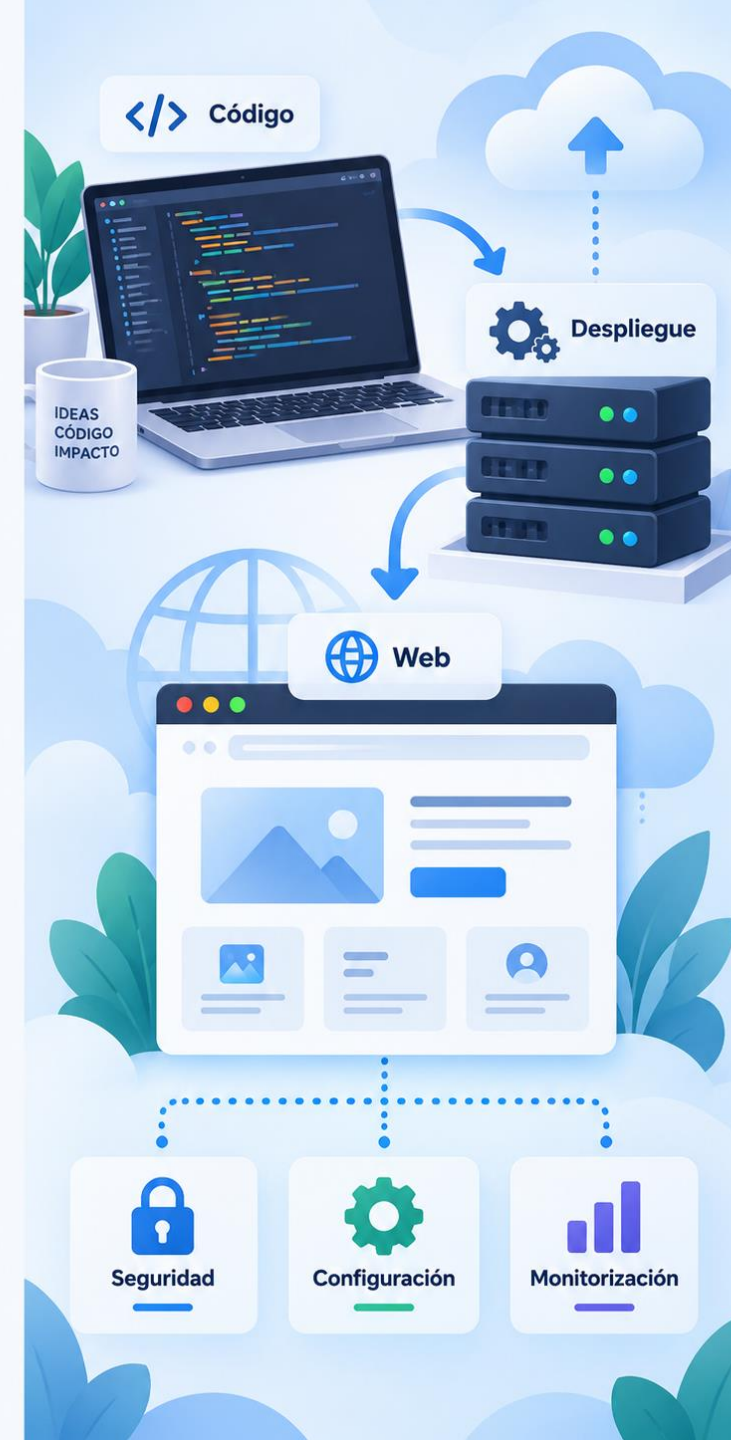


Control de versiones y documentación

Tema1 — Punto de partida



Objetivo de la sesión

Cambiar la pregunta: de “¿funciona?” a “¿se puede desplegar?”

Funciona en mi equipo

- Dependencias locales
- Datos y rutas personales
- Secretos mal ubicados
- Pasos recordados de memoria



Funciona para otros

- Proceso repetible
- Configuración externa
- Evidencias y diagnóstico
- Actualizaciones controladas

1

Registrar commits
como estados

2

Proteger qué no
debe entrar

3

Compartir GitHub y
remoto

4

Revisar ramas, PR y
checks

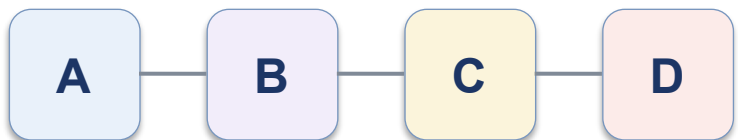
5

Identificar etiquetas
y versiones

Idea clave: el repositorio debe permitir reconstruir y explicar un estado desplegable.

Git registra estados, no copias de carpetas

Cada commit identifica un estado del proyecto y queda relacionado con los anteriores



A medida que haces commits, Git no guarda “una carpeta por versión”.
Guarda estados y su relación en el historial.

Qué nos interesa versionar en despliegue

- Código de aplicación
- Documentación y procedimientos
- Configuración reproducible
- Workflows y automatizaciones

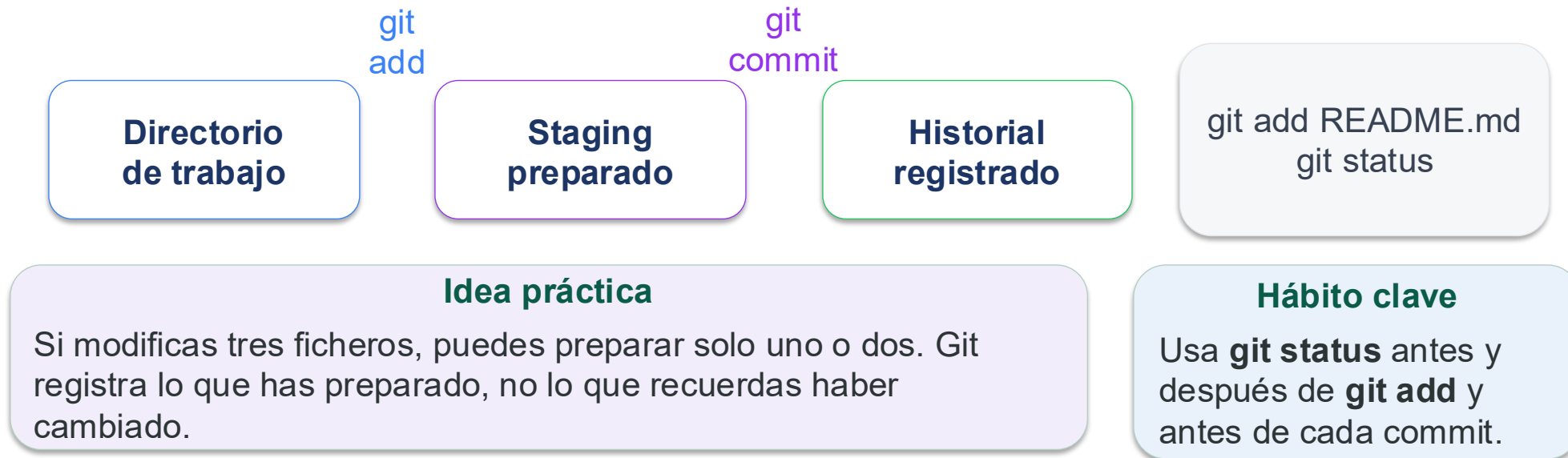
Idea útil

Un commit útil representa una intención concreta y deja una explicación mínima del cambio realizado.

En despliegue también versionamos documentación y configuración reproducible.

Del fichero editado al historial

Entre editar un fichero y registrarlo hay una zona intermedia: el staging



Reflejo básico: git status antes y después de git add.

Un commit debe representar una idea

Un historial útil debe poder leerse y entenderse.

Buenos mensajes

`docs: añade instrucciones de arranque`
`fix: corrige la ruta de almacenamiento`
`chore: configura variables de desarrollo`

Malos mensajes

`cambios`
`final`
`arreglo2`

feat:
funcionalidad

fix:
corrección

docs:
documentación

test:
pruebas

chore:
mantenimiento

Un buen commit permite entender por qué existe un cambio sin abrir todos los ficheros.

Decide qué entra antes del primer commit

Sí debe versionarse

- Código fuente
- README y documentación
- Configuración reproducible
- Workflows y automatización

Debe quedar fuera

- `target/`, `\*.class`
- `.idea/`, `.vscode/`
- `.env`, claves y tokens
- `uploads/` y datos locales

```
# Maven / Java  
target/  
*.class
```

```
# IDE  
.idea/  
.vscode/
```

```
# Secretos  
.env  
.env.*  
!.env.example
```

Consulta útil

```
git check-ignore -v ruta/al/fichero
```

La pregunta no es “¿puedo subirlo?”, sino “¿debe formar parte del historial?”.

.gitignore protege hacia delante

.gitignore

Evita nuevas incorporaciones.

Filtra lo local, generado o sensible.

Si ya estaba versionado

```
git rm --cached .env
```

Después, commit con la corrección.

Si era un secreto real

Se considera comprometido.
Debe revocarse o cambiarse.

`.env`
no se versiona

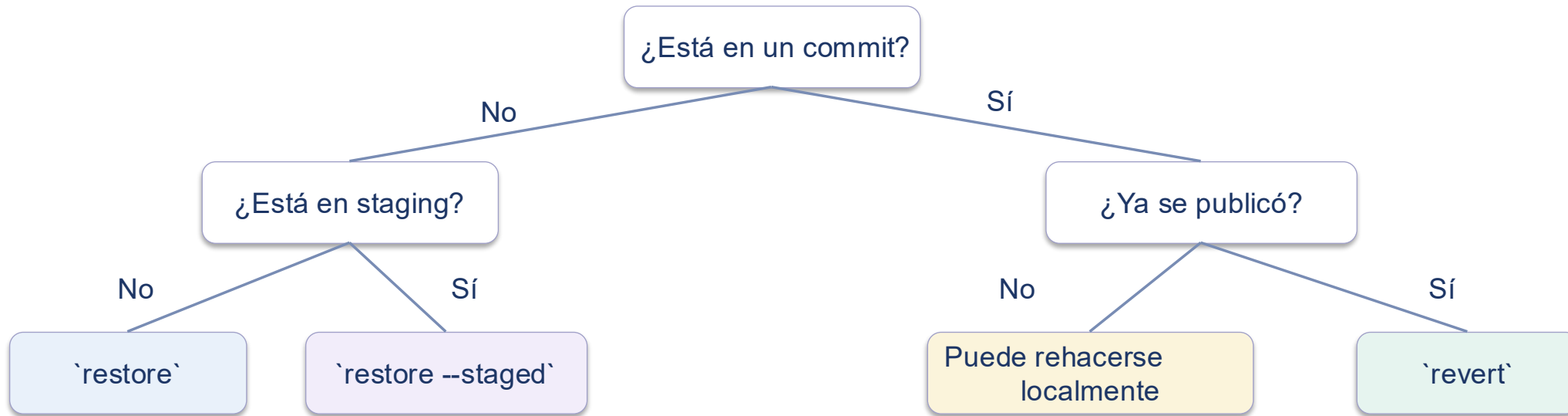
`.env.example`
sí se versiona

Configura
sin revelar

```
DB_HOST=bd
DB_NAME=escaparate
DB_USER=cambia-este-valor
DB_PASSWORD=cambia-este-valor
```

Borrar un secreto en un commit posterior no elimina la exposición anterior.

Deshacer: primero identifica dónde está el cambio



Pista práctica

Antes de deshacer, localiza si el cambio está en el directorio de trabajo, en staging o en un commit.

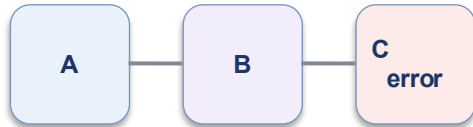
Regla segura

Error local y privado: puede rehacerse. Error ya publicado: normalmente se corrige hacia delante.

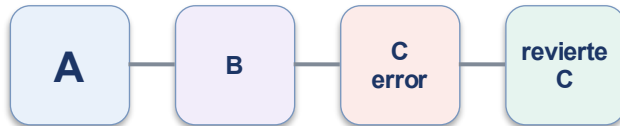
La pregunta clave: ¿es un cambio local o un historial compartido?

Un error publicado se corrige hacia delante

Antes



Después de `git revert C`



Qué conseguimos

- El estado actual queda corregido.
- El commit original sigue visible.
- La corrección queda documentada.

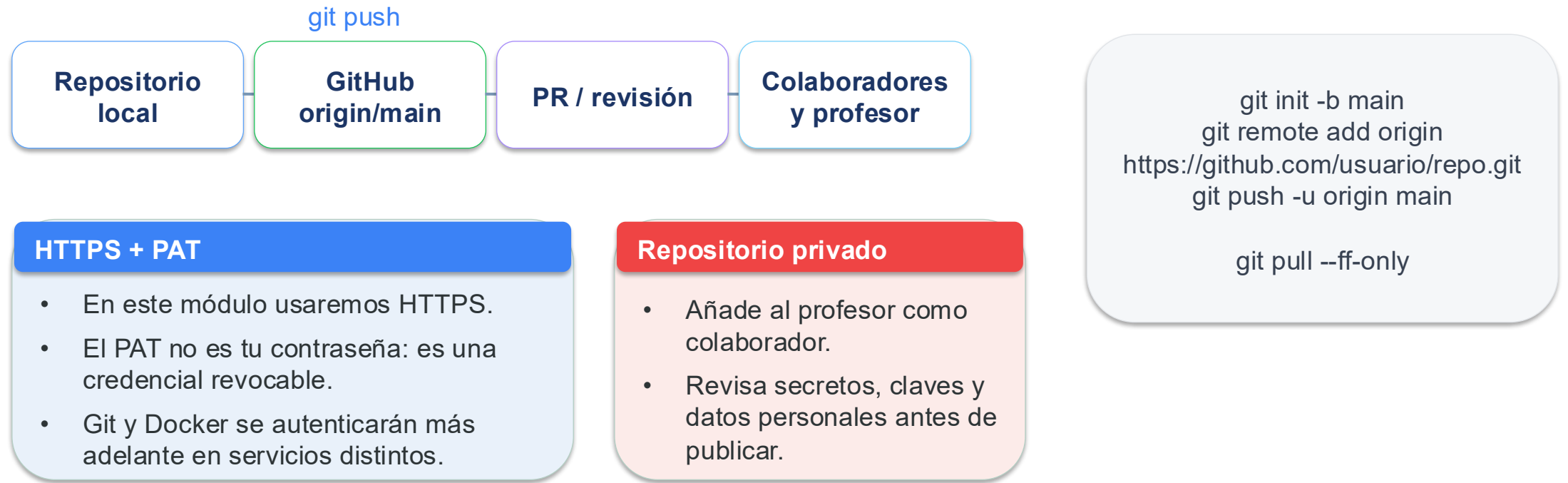
Qué debes recordar

`revert` es la opción habitual cuando el commit erróneo ya fue compartido.

La corrección pública queda registrada y no rompe el historial compartido.

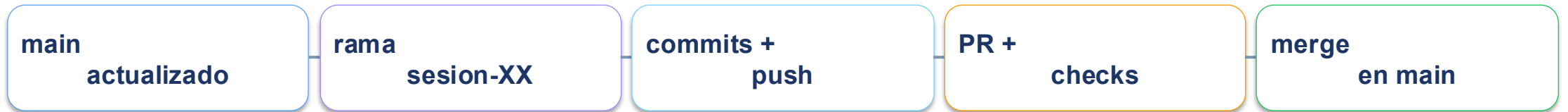
Del repositorio local a GitHub

Sincronización básica



En el aula el remoto será el punto común desde el que se revisará el trabajo.

Ramas, Pull Request y GitHub Actions



La Pull Request responde a tres preguntas

- Qué cambia
- Por qué cambia
- Cómo se ha comprobado

Las comprobaciones automáticas

- No sustituyen la revisión, pero impiden integrar estados claramente rotos.
- GitHub Actions ejecuta workflows guardados en ``.github/workflows/``.

```
git switch main  
git pull --ff-only  
git switch -c sesion-XX
```

```
git add <ficheros>  
git commit -m "docs: documenta la sesión"  
git push -u origin sesion-XX
```

main representa el último estado integrado que consideramos desplegable.

Referencias Git: qué se mueve y qué permanece

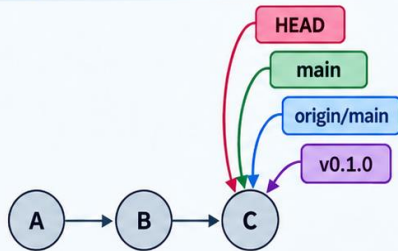
Git guarda snapshots, las referencias nos orientan.

Referencias Git: qué se mueve y qué permanece

HEAD, ramas y etiquetas

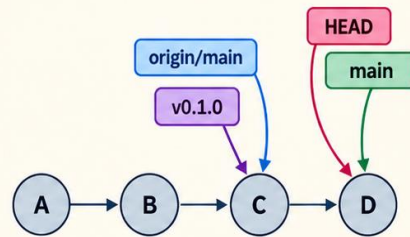
Misma historia, diferentes referencias

1 Antes del commit local



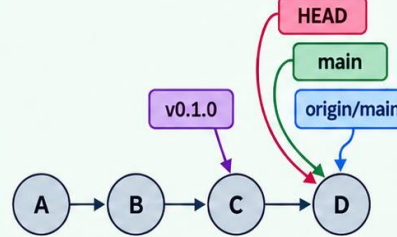
Todas las referencias apuntan al mismo estado.

2 Después de crear un commit local



El commit D existe solo en local hasta hacer push.

3 Después del push



La etiqueta no se mueve sola.



- 1 HEAD señala tu posición actual.
- 2 La rama actual (main en el ejemplo) avanza al crear un commit.
- 3 origin/main se actualiza al sincronizar con el remoto.
- 4 Una etiqueta fija marca un punto concreto del historial.

Leyenda de referencias

HEAD	Tu posición actual.
Rama	(ej. main) Rama local.
Rama remota	(ej. origin/main) Referencia al estado conocido de main en el remoto.
Etiqueta	(ej. v0.1.0) Punto fijo del historial.

Qué debes retener

- `HEAD` indica dónde estás trabajando.
- `main` es la rama local y avanza cuando haces nuevos commits sobre ella.
- `origin/main` representa el estado conocido del remoto.
- Una etiqueta como `v0.1.0` permanece fija hasta que la cambias deliberadamente.

```
git tag -a v0.1.0 -m "Primer estado identificable"
git push origin v0.1.0
```

Las ramas avanzan; las etiquetas identifican puntos concretos del historial.

Etiquetas y versionado semántico

Para qué sirve una etiqueta

- Identificar un estado estable
- Relacionar entregas o hitos
- Marcar una versión sin mover la rama

Ejemplos

`1.4.2 → 1.4.3` corrección
`1.4.2 → 1.5.0` nueva funcionalidad
`1.4.2 → 2.0.0` cambio incompatible

Versionado semántico

Parte	Cuándo cambia
MAYOR	cambios incompatibles
MENOR	nueva funcionalidad compatible
PARCHE	corrección compatible

En el módulo

La etiqueta `v0.1.0` identificará un primer estado útil del repositorio del curso. No tiene por qué coincidir con la versión interna de la aplicación.

Etiquetar no es “hacer una copia”: es poner nombre a un punto del historial.

La documentación también se versiona

Markdown esencial

- Título y subtítulos
- Listas
- Código en línea y bloques
- Enlaces relativos
- Imágenes

README útil

- Qué es el proyecto
- Requisitos
- Estructura del repositorio
- Flujo de trabajo
- Puesta en marcha y comprobación

Evita

- Rutas absolutas de tu equipo
- Copiar YAML o compose en varios lugares
- Guardar PAT, claves o secretos
- Convertir el README en un diario

El repositorio debe explicar cómo reconstruir el despliegue, no almacenar datos privados.

Actividad 1.2: tu repositorio como herramienta de despliegue



Al salir de la sesión deberías poder...

- Preparar el repositorio del curso como fuente de verdad.
- Proteger el histórico de estados que sí deben conservarse.
- Crear una rama, una PR, checks y una primera etiqueta.
- Distinguir entre versionar información útil y publicar secretos o ruido.

Próxima sesión: empezaremos a empaquetar piezas con contenedores.

Cierre de la sesión

Lo mínimo que debe quedar claro

- ✓ Git registra estados; el flujo básico es directorio de trabajo → staging → commit.
- ✓ .gitignore evita nuevas incorporaciones, pero no borra el pasado.
- ✓ Un secreto publicado debe revocarse o cambiarse.
- ✓ En el módulo trabajaremos con rama corta → Pull Request → comprobaciones → main.
- ✓ Las ramas avanzan; las etiquetas identifican estados concretos.
- ✓ La documentación también forma parte del despliegue reproducible.

Próxima sesión: el repositorio se convertirá en la base del empaquetado con contenedores.